Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

НА ТЕМУ:

Разработка консольного приложения с применением ООП

Студент	ИУ5-23Б		Е. П. Балаева
	(группа)	(подпись, дата)	(И.О. Фамилия)
Руководитель курсовой работы			Ю. Е. Гапанюк
		(подпись, дата)	(И.О. Фамилия)

2026 г.

1. Постановка задачи

Целью работы является разработка консольного приложения на языке C# с применением принципов объектно-ориентированного программирования (ООП) и интеграцией с базой данных SQLite.

Согласно варианту №3, объектами предметной области являются: * **Марки автомобилей** (справочник); * **Автомобили** (основные данные).

2. Проектирование классов-моделей

Классы-модели реализуют инкапсуляцию данных. В классе `Car` реализована бизнес-логика проверки корректности входных данных (мощность двигателя).

```
public class Car
{
    public int Id { get; set; }
    public int BrandId { get; set; }
    public string ModelName { get; set; }

    private int _horsepower;
    public int Horsepower
    {
        get => _horsepower;
        set
        {
            // Валидация согласно требованиям
            if (value < 0)
                throw new ArgumentException("Мощность не может быть меньше 0");
            _horsepower = value;
        }
    }

    public Car(int id, int brandId, string modelName, int horsepower)
    {
        Id = id;
        BrandId = brandId;
        ModelName = modelName;
        Horsepower = horsepower; // Срабатывает сеттер с валидацией
    }

    public override string ToString() =>
        $"ID: {Id} | Модель: {ModelName} | Марка (ID): {BrandId} | Мощность:
{Horsepower} л.с.";
}
```

3. Работа с базой данных (DatabaseManager)

Класс `DatabaseManager` инкапсулирует логику ADO.NET для работы с SQLite. Реализована поддержка внешних ключей и автоматического импорта данных.

```
public void InitializeDatabase()
{
    using var conn = new SqliteConnection(_connectionString);
    conn.Open();
    var cmd = conn.CreateCommand();
    cmd.CommandText = @"
        CREATE TABLE IF NOT EXISTS brands (
            brand_id INTEGER PRIMARY KEY,
            brand_name TEXT NOT NULL
        );
        CREATE TABLE IF NOT EXISTS cars (
            car_id INTEGER PRIMARY KEY AUTOINCREMENT,
            brand_id INTEGER NOT NULL,
            model_name TEXT NOT NULL,
            horsepower INTEGER NOT NULL,
            FOREIGN KEY (brand_id) REFERENCES brands(brand_id)
        );";
    cmd.ExecuteNonQuery();
}
```

3.1. Импорт данных из CSV

Метод `ImportFromCsv` обеспечивает начальное заполнение базы данных, используя инструкцию `INSERT OR IGNORE` для предотвращения дублирования записей при повторных запусках.

```
public void ImportFromCsv(string brandsPath, string carsPath)
{
    using var conn = new SqliteConnection(_connectionString);
    conn.Open();

    // Чтение и парсинг CSV с разделителем ';'
    if (File.Exists(brandsPath))
    {
        var lines = File.ReadAllLines(brandsPath).Skip(1);
        foreach (var line in lines)
        {
            var p = line.Split(';');
            var cmd = conn.CreateCommand();
            cmd.CommandText = "INSERT OR IGNORE INTO brands (brand_id, brand_name)
```

```
VALUES (@id, @n)";
            cmd.Parameters.AddWithValue("@id", p[0]);
            cmd.Parameters.AddWithValue("@n", p[1]);
            cmd.ExecuteNonQuery();
        }
    }
}
```

4. Реализация отчетов (ReportBuilder)

Применен паттерн **Fluent Interface** для гибкого построения табличных отчетов в консоли.

```
public class ReportBuilder
{
    // ... поля настройки ...
    public string Build()
    {
        var (columns, rows) = _db.ExecuteQuery(_sql);
        var sb = new StringBuilder();

        // Отрисовка заголовков и разделителей
        for (int i = 0; i < columns.Length; i++)
            sb.Append(columns[i].PadRight(_widths[i]));

        sb.AppendLine().AppendLine(new string('-', _widths.Sum()));

        // Отрисовка строк данных
        foreach (var row in rows)
        {
            for (int i = 0; i < row.Length; i++)
                sb.Append(row[i].PadRight(_widths[i]));
            sb.AppendLine();
        }
        return sb.ToString();
    }
}
```

5. Пример работы программы

Программа предоставляет пользователю интерактивное меню. Ниже представлен пример вывода сложного отчета с объединением таблиц (JOIN).

```
=== АВТОМОБИЛИ ПО МАРКАМ ===
Модель               Марка           Мощность (л.с.)
-----------------------------------------------------
Camry                Toyota          181
```

```
M5                  BMW             600
Vesta               Lada            106
```

Заключение

В ходе выполнения ИДЗ была спроектирована архитектура приложения, разделяющая логику хранения данных, бизнес-модели и интерфейс представления. Использование SQLite позволило реализовать полноценную работу с реляционными данными, включая контроль целостности через внешние ключи.